Course Code : MAT338

Semester: VIII

L	T	P	C
4	0	2	5

NEURAL NETWORKS & DEEP LEARNING

Course Objective:

This course provides a rigorous introduction to neural networks and deep learning. It covers the mathematical foundations of artificial neural networks, including the geometry of high-dimensional spaces, optimization algorithms for training (gradient descent and back propagation), and regularization techniques. Students will study various network architectures—from simple feed-forward networks and multilayer perceptron's to convolutional neural networks—and learn how these models approximate complex functions. The focus is on understanding why and how deep networks work from a theoretical perspective, alongside practical implementation.

UNIT I: Fundamentals of Neural Networks 15 Periods

Neuron model and biological inspiration. Formal definition of an artificial neuron; activation functions (sigmoid, ReLU, tanh and their characteristics). Perceptron learning rule and **logistic regression** as a single-layer neural network. **Loss functions** (mean squared error, cross-entropy) for regression and classification. Introduction to computational graphs for representing model computations.

UNIT II: Training Deep Networks 15 Periods

Multilayer Perceptrons (MLPs) and network architectures. Back propagation algorithm derivation: chain rule of calculus applied to compute gradients in layered networks. Gradient descent variants (batch vs. stochastic vs. mini-batch) and their mathematical convergence properties. Initialization strategies and their effect on training (Xavier/He initialization). Practice problem: derive backprop for a 2-layer MLP by hand for a simple dataset.

UNIT III: Optimization and Regularization 15 Periods

Deeper dive into optimization algorithms: Momentum, Nesterov accelerated gradient, RMSProp, Adam – understanding the math behind each optimizer and when to use them. **Learning rate** scheduling and tuning. Problems of vanishing/exploding gradients in deep networks; solutions like activation function choice and batch normalization (mean-variance normalization across mini-batch).**Regularization techniques:** L1/L2 weight penalties (connection to sparsity and ridge regression), dropout (with probabilistic interpretation), and data augmentation.Bias–variance trade-off in deep learning models.

UNIT IV: Convolutional Neural Networks (CNNs) 15 Periods

Motivation for convolution (parameter sharing for grid-structured data like images). Convolution operation as cross-correlation; kernels/filters and feature maps. **Pooling** layers and their effect on translational invariance. Example CNN architectures (LeNet, AlexNet) and their breakthroughs. Mathematical analysis of convolution (dimensions, stride, padding) and the effect on output sizes.Concept of receptive fields.

UNIT V: Training Deep Learning Models & Case Studies 15 Periods

End-to-end training pipeline: data preprocessing, weight initialization, forward pass, back propagation, and evaluation metrics (accuracy, precision/recall for classification). Discussion of hardware considerations (GPU vs CPU for large matrix operations). Case study: build and train a small deep network for a practical task (e.g., handwritten digit classification or simple image recognition). Analyze training curves (learning curves) to diagnose issues. Introduction to using a framework (PyTorch or TensorFlow) – defining models, auto-differentiation, and debugging common errors. This unit consolidates the theory in a practical context, preparing students for more complex architectures in subsequent courses.

Laboratory: Deep Learning Lab – In the 2-hour weekly lab, students implement neural networks both from scratch and using deep learning libraries. Early labs focus on coding a multilayer perceptron for a small dataset without high-level library support (to appreciate the math of back propagation). Later labs involve using PyTorch / TensorFlow to experiment with CNNs. Sample lab assignments: implement and visualize different activation functions; train a 3-layer MLP on a toy dataset using only numpy (manual backprop); use PyTorch to build a CNN for CIFAR-10 dataset and experiment with hyper parameters. One lab will involve reading a research paper excerpt (e.g. a section of a seminal Deep Learning paper) and relating it to the implemented models, to build scholarly understanding. Lab evaluation will include coding style, result analysis, and a short report on findings for each experiment.

TEXTBOOKS

1. Goodfellow, Ian, *Deep Learning*, MIT Press, 1st Edition, 2016.
2. Nielsen, Michael A., *Neural Networks and Deep Learning*, Determination Press (Online Publication), 1st Edition, 2015.
3. Rumelhart, David E., *Learning Representations by Back-Propagating Errors*, Nature Publishing Group, Original Research Paper, 1986.
4. Li, Fei-Fei (et al., Stanford University), *CS231n: Convolutional Neural Networks for Visual Recognition*, Stanford University Online Notes, Course Edition, Latest Revision 2017–2018 (commonly cited).
5. Indian Institute of Science (IISc), *Pattern Recognition and Neural Networks (Course Material)*, Department of Mathematics & Computing, Elective Course Reference, Current Curriculum.

ONLINE MATERIALS

1, https://nptel.ac.in/courses/117105084

2. https://onlinecourses.nptel.ac.in/noc20_cs62/preview

Course Outcomes:

On successful completion, students will be able to:

Unit I	• Explain the core concepts of neural networks (weights, biases, activation functions) and the universal approximation capability of multi-layer networks
Unit II	• Derive and apply the back propagation algorithm for computing gradients in deep networks, understanding the role of calculus in updating parameters
Unit III	• Analyze the training process in terms of loss landscapes and optimization, comparing algorithms like stochastic gradient descent, momentum, and Adam for convergence behavior.
Unit IV	• Implement neural networks from scratch and using a deep learning framework (PyTorch/TensorFlow), and evaluate their performance on benchmark datasets (e.g., MNIST image classification)..
Unit V	• Discuss issues such as overfitting and generalization, and apply regularization techniques (L2 norm, dropout, batch normalization) to improve network generalization.